COSC 2637/2633 Big Data Processing
Assignment 1
MapReduce-Based Taxi Fleet and Company Performance Analysis

Assessment Type	– This assignment may be completed **individually** or in **groups of two or three** students. – Submit online via Canvas → Assignment 1. – Marks awarded for meeting requirements as closely as possible. – Clarifications/updates may be made via announcements or relevant discussion forums.
Due Date	Due at 23:59, 30/08/2026, Sunday
Marks	25

Overview

Develop and implement MapReduce programs that demonstrate your understanding of MapReduce principles by solving complex data processing problems on the Hadoop execution platform.

Learning Outcomes

The key course learning outcomes are:

- CLO 1: model and implement efficient big data solutions for various application areas using appropriately selected algorithms and data structures.
- CLO 2: analyse methods and algorithms, to compare and evaluate them with respect to time and space requirements and make appropriate design choices when solving real-world problems.
- CLO 3: motivate and explain trade-offs in big data processing technique design and analysis in written and oral form.
- CLO 4: explain the Big Data Fundamentals, including the evolution of Big Data, the characteristics of Big Data and the challenges introduced.
- CLO 6: apply the novel architectures and platforms introduced for big data.

Group Assessment

You may complete this assignment either individually or in a group of two or three students. No other group sizes are permitted. Groups must consist of students who are enrolled at the same level (either all Undergraduate COSC2633 students or all Postgraduate COSC2637 students). All members are expected to contribute equally. Unless a contribution dispute is reported (and supported by the work log), all group members will receive the same grade. Individual work is allowed but not encouraged. Please note that no bonus marks will be awarded for completing the work individually.

Group Formation and Locking Policy

- Navigate to **People** in Canvas, then select the **Groups** tab.
- Choose an empty **A1 group** and join it along with your group members.
- **You are not permitted to create new groups. You are not permitted to rename the groups.** This restriction is in place because student-created groups are not linked to the grading system, which means they cannot be identified by the system during marking.
- Canvas groups are intended only for students completing the assignment in groups of two or three. Students completing the assignment individually do not need to join a Canvas group.
- Groups will be locked 72 hours before the deadline. Once groups are locked, students can no longer join, leave, or modify their groups. This ensures consistency in grading and prevents last-minute changes that could affect group accountability.

Before submitting A1, you must:

1. Ensure your group membership is correct.
2. Verify that all group members are listed in the Canvas A1 group.

3. Submit the assignment only after groups have been finalised and locked.

Please DO NOT submit your assignment before your group has been correctly formed and locked.
You are responsible for forming your group before beginning the assessment. Students who do not correctly form their group before submission will receive a 10% deduction (2.5 marks).

Group Leadership

The first student to join a group on Canvas is assigned as the group leader by default. However, this role can be changed later within the group settings if needed. The group leader is responsible for ensuring that the group is correctly finalised before the deadline. This includes:

- Confirming all required files are prepared and correctly named.
- Verifying the correct members are listed in the group.
- Submitting the final work before the deadline.
- Keeping and submitting the group work log as part of the submission.

The group leader is responsible for coordinating communication and organising weekly meetings. The role does not provide additional authority over other group members, nor does it make the group leader responsible for completing the assignment on behalf of the group or for the actions of other members. All group members share equal responsibility for contributing to the assignment, reviewing the final submission, and complying with the course's academic integrity requirements.

Extension Policy for Group Assignments

Any extension requests for group assignments must be approved by the Course Coordinator. Group extensions will only be considered under exceptional circumstances (**if more than 50% of the group members are unable to contribute at the same time**) and require supporting documentation.

Please note that extensions are granted only for **unexpected circumstances outside a student's control**, as outlined in RMIT policy. Regular commitments such as part-time work or general workload are considered foreseeable and therefore do not typically meet the criteria. Students are expected to plan their work and make every effort to meet the original deadline. The following reasons are **generally not considered valid** for granting an extension, as they are foreseeable or within a student's control:

- **Academic workload & time management:** multiple assignments due in the same week; general study pressure or heavy workload; poor time management or starting late; and/or underestimating the difficulty of the assignment.
- **Employment commitments:** regular part-time or casual work shifts; busy work schedule or increased workload, and/or choosing to take additional shifts.
- **Travel & personal plan:** pre-booked travel, holidays, or social commitments; events arranged in advance; and/or planned relocation or return home arrangements.
- **Group-related issues:** difficulty coordinating with group members; miscommunication within the group; one member being unavailable or unresponsive; and/or late group formation.
- **Technical & submission issues (avoidable):** uploading incorrect files; last-minute technical problems due to late submission; Internet issues at the time of submission (if preventable); and/or failure to check submission requirements.
- **Minor or foreseeable circumstances:** feeling stressed or overwhelmed (without supporting evidence); general tiredness or lack of preparation, and/or known commitments that were planned in advance.

If you wish to apply for an **individual extension**, you must withdraw from your group (if applicable) before the assignment deadline. You will then be required to complete and submit the entire assignment individually. If your individual application is approved, alternative tasks will be given only to you — not to your former group members.

What to Do If a Team Member Isn't Contributing

- Try to resolve the issue first through communication within the group. Document communication attempts (e.g., emails, messages). Keep a simple log of who completed which parts of the work.
- If the situation doesn't improve, you may choose to leave the group, join other groups OR complete the assignment individually.

Please notify the tutor and the course coordinator early if you decide to do so. Adjustments due to non-contributing members will not be granted unless clear evidence is provided.

Assessment Details

You have two datasets: `Taxis.txt`, which contains taxi information, and `Trips.txt`, which contains trip records. Both files are stored in the `/Input` directory on HDFS.

A sample of `Taxis.txt`	A sample of `Trips.txt`
Taxi ID, company ID, model, year	**Trip ID, Taxi ID, fare, distance, pickup_x, pickup_y, dropoff_x, dropoff_y**
470,0,80,2018	0,354,232.64,127.23,46.069,85.566,10.355,4.83
332,11,88,2013	1,173,283.7,150.74,5.02,31.765,88.386,27.265
254,10,62,2018	2,8,83.84,43.17,63.269,33.156,92.953,60.647
460,4,90,2022	3,340,259.2,136.3,14.729,13.356,14.304,90.273
113,6,23,2015	4,32,270.07,152.65,27.965,13.37,77.925,62.82
275,16,13,2015	5,64,378.31,202.95,1.145,94.519,98.296,35.469
318,14,46,2014	6,480,235.98,121.23,66.982,66.912,5.02,31.765

- Complete the following MapReduce programming tasks **with Python and the methods taught in this course only**. You must apply the knowledge and skills acquired in this course to manage your HDFS directories.
- Using any other language like Java will directly lead to a 0 mark on the assignment.
- **You are not allowed to use any Python MapReduce library such as `mrjob`.**
- A reasonable big data processing program should not create a data structure to hold the complete data in memory. The data should be processed line by line, and the intermediate results are stored in memory only if it is the method taught in the course, like in-memory combining. Reducers should mainly focus on final aggregation of already partially processed data. Heavy processing should be pushed to the mapper or handled using combiners to reduce network and memory overhead.
- You are required to include a step-by-step explanation of your MapReduce job design in your group/individual work log Section 2. **You are not required to include any Python code in this section.** The purpose of this section is to explain your design and reasoning. A clear description, tables, examples of `<key, value>` pairs, diagrams, or flowcharts are encouraged if they help illustrate how data moves through your solution.
  - Providing a clear description of the data flow is an essential part of this assignment. Your explanation should demonstrate your understanding of how data is transformed and passed through each stage of the MapReduce pipeline, rather than simply presenting the final code. In particular, you must explain the input and output formats of every mapper and reducer, how each `<key,value>` pair is generated and consumed. If a custom partitioner or comparator is used, you must clearly describe its purpose, configuration, and effect on the execution.

    Note: Insufficient or unclear explanations may make it difficult to assess your understanding of the MapReduce design and could result in mark deductions, even if the program produces the correct output.
  - If your solution uses multiple MapReduce jobs, you must clearly explain how data is passed from one job to the next. In particular, describe the output format produced by the reducers of the previous job, where that output is stored, and how it becomes the input to the mapper of the subsequent job. Your explanation should trace the complete data flow across all jobs, showing how intermediate `<key, value>` pairs are transformed and reused.

    Note: Simply stating that one job feeds into another is insufficient; you must explain the exact format and purpose of the intermediate data exchanged between jobs.

Task 1 – Taxi-Level Trip Efficiency Analysis (5 marks)

(Task for BOTH Undergraduate COSC2633 and Postgraduate COSC2637)

A city transport authority is analysing historical taxi trip data to better understand how individual taxis are used across different types of journeys. Instead of only evaluating taxi companies at a high level, the authority wants to examine **taxi-level operating patterns**, including whether each taxi is mainly used for short, medium, or long-distance trips. This analysis can help the decision-making team understand taxi utilisation patterns, compare fare behaviour across different trip types, identify unusual fare patterns, and support decisions about fleet allocation, service planning, and operational monitoring.

To support operational decision making, the authority needs a summary for each taxi and each trip-distance category. Trips are classified into three types: **long trips** with a distance of **200 or more**, **medium trips** with **100 or more but less than 200**, and **short trips** with **less than 100**.

For each combination of **taxi ID** and **trip type**, the authority wants to compute the total number of trips, the maximum fare, the minimum fare, and the average fare per trip. The combination of **taxi ID** and **trip type** should be treated as the primary key in the final output.

Because the dataset is large, the task must be implemented using MapReduce. Your program should implement **in-mapper combining** with state preserved across input lines to maximise the effectiveness and efficiency of the map phase.

HDFS Input/Output Directory Structure

You must follow the exact naming conventions for all input and output directories and files on HDFS.

- /Input/Trips.txt
- /Output/task1

Directory and file names are case-sensitive. You must use names like "task1" and not "Task1" or "TASK1". Failure to follow this naming convention may result in execution and grading issues.

Output Format Requirements

Your output must contain exactly 6 fields, separated by a tab (\t) character, in the following order:

1. taxi ID
2. trip type (*long*, *medium*, or *short*)
3. the total number of trips (integer)
4. the maximum fare (decimal with two places)
5. the minimum fare (decimal with two places)
6. the average fare per trip (decimal with two places)

Each line of your output should follow this format **exactly** to ensure consistency and correct evaluation.

Execution Requirements

- Your code must support execution with 3 reducers. Submissions that do not comply with this reducer requirement will be subject to a significant mark deduction.
- You need to submit a shell script named `task1-run.sh`. When executed (`./task1-run.sh`), the shell script should perform the full task end-to-end, without any external or manual intervention.

Task 2 – Clustering Taxi Trips by Drop-off Location (10 marks)

(Task for BOTH Undergraduate COSC2633 and Postgraduate COSC2637)

You are asked to write a MapReduce program with Python to cluster trips in `Trips.txt` based on the **drop-off locations**. This information can be used to identify popular destination zones, optimize driver allocation, and support location-based marketing or infrastructure planning based on clustering patterns in drop-off areas.

Your code should implement ***k*–medoid** clustering algorithm known as **Partitioning Around Medoids** (PAM) algorithm which is described below:

1. **Initialize:** randomly select k of the n data points as the medoids.
2. **Assignment**: Associate each data point to the closest medoid.
3. **Update**: For each medoid m and each data point o associated with m, **swap** m and o, and compute the total cost of the configuration (that is, the average dissimilarity[1] of o to all the data points associated to m). The candidate o that results in the lowest configuration cost after the swap becomes the new medoid. (**Note: The new medoid must be obtained through this swap evaluation process**.)
4. **Iteration:** Repeatedly alternating steps 2 and 3 until there is no change in the assignments or after a given number v of iterations.

The code must work for 3 reducers, for different settings of k, and for different settings of v. The value of v and the initial k data points are input of the program using a separate file named as `initialization.txt`. An example of the file for $k = 3$ and $v = 10$ looks like:

```
10
85.679   99.074
11.737   11.615
83.802    1.277
```

Note: your program should **read and use those given medoids directly**, you do **not** need to randomly choose medoids from the dataset during the **first step (initialize)**.

HDFS Input/Output Directory Structure

You must follow the exact naming conventions for all input and output directories and files on HDFS.

- /Input/Trips.txt
- /Output/task2

Directory and file names are case-sensitive. You must use names like "task2" and not "Task2" or "TASK2". Failure to follow this naming convention may result in execution and grading issues.

Iteration Output Requirement

During each iteration of the PAM process, your program must print:

1. iteration number; and
2. for each cluster: the medoid coordinates (`medoid_x` and `medoid_y`), the number of assigned data points (`#points` in integer), and the average dissimilarity between the medoid and the data points in that cluster (`avg_dissimilarity` in decimal with two places).

Note: This standard output (directly to the terminal) at each iteration is important for tracking the progress of the algorithm and verifying convergence behavior.

[1] Using Euclidean distance between two points as the dissimilarity measure.

Final Output Format Requirements

The output of task 2 should contain the following fields generated by your program.
Assume $k = 3$, when marker runs the following command:

```
hadoop fs -getmerge /Output/task2/part* task2_output.txt
```

Your resulting `task2_output.txt` file must contain exactly 3 lines, with each representing one cluster.

```
medoid0_x    medoid0_y    #points0    avg_dissimilarity0
medoid1_x    medoid1_y    #points1    avg_dissimilarity1
medoid2_x    medoid2_y    #points2    avg_dissimilarity2
```

Execution Requirements

- Your code should work for different settings of k and v.
- Your code must support execution with 3 reducers. Submissions that do not comply with this reducer requirement will be subject to a significant mark deduction.
- You need to submit a shell script named `task2-run.sh`. When executed (`./task2-run.sh`), the shell script should perform the full task end-to-end, without any external or manual intervention.
- Please assume that the provided `initialization.txt` file (in the specified format) is in the same directory as all your Python files and your Bash script. Your program should reference this file directly without relying on external or absolute paths.
- During each iteration of the PAM process, you are responsible for managing the HDFS output directories appropriately. Since Hadoop does not allow a job to write to an existing output directory, your program should ensure that a valid output directory is used for each iteration.
- **Only the final output stored in the specified HDFS folder will be assessed.** Any intermediate results produced during execution will not be marked. You are expected to manage intermediate outputs sensibly (e.g., keep them out of the `/Output/task2` directories, and **clean them up after the job is done to avoid confusion or storage issues**).

Task 3 – Company Performance Analysis (10 marks)

(Task for Postgraduate COSC2637 ONLY)

You are required to use what you learned so far to solve a slightly more advanced task.

The transport authority wants to evaluate **which taxi companies operate most efficiently across the city.** To provide this information and support decision making, you are required to write a MapReduce program in Python to compute the following information for each **taxi company**:

1. total revenue (calculated as the sum of fares),
2. total number of trips,
3. fleet size (calculated as the number of distinct taxis)
4. revenue per taxi (calculated as total revenue divided by fleet size), and
5. average trip distance.

Note: this task must be implemented using three separate MapReduce jobs:

- Job 1 – **Join**: Combine the relevant fields from the input datasets.
- Job 2 – **Aggregation**: Compute the required metrics **for each taxi company**, including total revenue, total trips, fleet size, revenue per taxi, and average trip distance.
- Job 3 – **Sorting**: Sort the results based on the specified criterion, namely total revenue in descending order.

The execution of the three jobs should be specified in `task3-run.sh`. Sorting must be performed within the MapReduce program. You are not permitted to sort the final output using a Bash script or any post-processing command after the MapReduce job has completed.

Note: The use of `TotalOrderPartitioner` for Job 3 Sorting is not permitted. You are expected to solve this task using only the techniques and components covered in this course. Think about how to design your intermediate keys and how to control the distribution of data across the **3 reducers** using the MapReduce mechanisms introduced in the course.

HDFS Input/Output Directory Structure

You must follow the exact naming conventions for all input and output directories and files on HDFS.

- /Input/Trips.txt
- /Input/Taxis.txt
- /Output/task3

Directory and file names are case-sensitive. You must use names like "task3" and not "Task3" or "TASK3". Failure to follow this naming convention may result in execution and grading issues.

Output Format Requirements

When marker runs the following command:

```
hadoop fs -getmerge /Output/task3/part* task3_output.txt
```

Your resulting `task3_output.txt` file must meet the following criteria:

Your output must contain exactly 6 fields, separated by a tab (\t) character, in the following order:

1. Company ID
2. total revenue (decimal with two places)
3. the total number of trips (integer)
4. fleet size (integer)
5. revenue per taxi (decimal with two places)
6. average trip distance (decimal with two places)

Each line of your output should follow this format **exactly** to ensure consistency and correct evaluation.

The output must be sorted in **descending** order based on `total revenue`.
Only the final output stored in the specified HDFS folder will be assessed.

Any intermediate results produced during execution will not be marked. You are expected to manage intermediate outputs sensibly (e.g., keep them out of the `/Output/task3` directories, and **clean them up after the job is done to avoid confusion or storage issues**).

Execution Requirements

- **You are required to use multiple MapReduce jobs to complete this task.** Each job **must be configured to run with exactly 3 reducers** to demonstrate parallel processing and proper partitioning of the data. Submissions that do not comply with this reducer requirement will be subject to a significant mark deduction.
- The execution flow of these jobs must be specified in your `task3-run.sh` script. This script should correctly chain the jobs together, ensuring that the output of one job is properly passed as the input to the next, without any external or manual intervention.
- **Note: It is illegal to copy `Trips.txt` and/or `Taxis.txt` to the local machine and process them.**

Task 4 – (10 marks)

(Task for Undergraduate COSC2633 ONLY)

A standard deviation (denoted as σ) is a measure of how dispersed the data is in relation to the mean. A low standard deviation means data are clustered around the mean, and a high standard deviation indicates data are more spread out. For example, low variance drivers tend to serve specific areas or trip lengths (e.g., airport shuttles, city loops); high variance drivers handle diverse routes — ideal for flexible scheduling or peak-time deployment. This information can be used to strategically assign drivers based on their historical trip profiles, improving efficiency and better aligning drivers with demand patterns.

To calculate the standard deviation, use the following formula:

$$\sigma = \sqrt{\frac{\sum_{i=1}^{N} |x_i - \mu|^2}{N}}$$

In this formula, σ is the standard deviation, x_i is a data value, μ is the mean, and N is the total number of data points. Write a python-based MapReduce solution to calculate the mean and standard deviation of trip distances per taxi. **To calculate the standard deviation, you must implement the formula provided in the assignment instructions. Alternative formulas or approximations (such as Welford's algorithm or population-based shortcuts) are not permitted.** Submissions that use an alternative approach may receive **up to 50% of the total marks available for that task**, even if the final numerical results are correct.

HDFS Input/Output Directory Structure

You must follow the exact naming conventions for all input and output directories and files on HDFS.

- /Input/Trips.txt
- /Output/task4

Directory and file names are case-sensitive. You must use names like "task4" and not "Task4" or "TASK4". Failure to follow this naming convention may result in execution or grading issues.

Output Format Requirements

Your output must contain exactly 3 fields, separated by a tab (\t) character, in the following order:

1. taxi#
2. μ: the mean of trip distances (decimal with two places)
3. σ: the standard deviation of trip distances (decimal with two places)

Each line of your output should follow this format **exactly** to ensure consistency and correct evaluation.

Execution Requirements

- **You are required to use a single MapReduce job to complete this task.** The task is specifically designed to assess your ability to solve the problem within a single MapReduce workflow. Submissions that use multiple MapReduce jobs may still receive partial credit (up to 75% of the marks allocated to this task) if the output is correct; however, they will not be eligible for full marks.
- The job **must run with exactly 3 reducers** to ensure proper use of parallel processing. Submissions that do not comply with this reducer requirement will be subject to a significant mark deduction.
- You need to submit a shell script named `task4-run.sh`. When executed (`./task4-run.sh`), the shell script should perform the full task end-to-end, without any external or manual intervention.

Note: Storing all distances per taxi in an in-memory array (e.g., distances = []**) within the reducer does not scale and undermines the principles of MapReduce.** Submissions that use this approach may produce correct results on small datasets; however, they do not demonstrate an appropriate MapReduce design. Therefore, such submissions will receive up to 25% of the marks allocated to this task, even if the numerical output is correct.

Submission Instructions

Your assignment should follow the requirement below and submit via Canvas > Assignment 1. Assessment declaration: when you submit work electronically, you agree to the assessment declaration.

Failure to follow the format requirements incurs up to 10 marks penalty.

1. Prepare your zip file

- Place all required files into a single folder, including Python code files (*.py) and Bash shell scripts (*.sh).
- **Do NOT include any subfolders.**
- Compress the folder into a zip file.

2. Prepare a pdf file for Turnitin plagiarism check

- Copy and paste the code and shell scripts from **all three tasks** into a plain text editor.
- Save this as a **pdf file**.
- **The PDF must be uploaded as an individual file, not included inside the ZIP file,** to ensure it is correctly processed by Turnitin for plagiarism checking.

Please note that, in accordance with RMIT policy, we cannot mark a submission that has not been processed through the required plagiarism checking system. Failure to submit the PDF separately will result in your work being unable to be assessed.

3. Prepare your work log doc file

- Please double-check that you have completed all three sections.
- Use the **Submission Declaration Checklist** to verify that your submission satisfies all submission requirements before submitting.

Name your zip file and pdf file using the following format:

`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.zip`
`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.pdf`
`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.doc`

Replace:

- `GroupLeaderStudentID` with the student number of the group leader
- `GroupNumber` with your group number (e.g., 3)
- `PG` if you are in a postgraduate program; `UG` if you are in an undergraduate program

Example: `s1234567_Group3_UG.zip` and `s1234567_Group3_UG.pdf`

The group leader is responsible for submitting the group's final zip file. Duplicate submissions from the same group will overwrite the original submission.

If you are completing A1 individually, name your zip file using the following format:

`<StudentID>_<UG/PG>.zip`
`<StudentID>_<UG/PG>.pdf`
`<StudentID>_<UG/PG>.doc`

Replace:

- `StudentID` with your actual student number
- `PG` if you are in a postgraduate program; `UG` if you are in an undergraduate program

Example**:** `s1234567_UG.zip` and `s1234567_UG.pdf`

Note: Both group submissions and individual submissions are required to include a work log. For group submissions, the work log must include a member contribution section (Section 1) that clearly describes each member's contributions. For individual submissions, a work log is still required; however, the member contribution section is not applicable and does not need to be completed.

Important

- You must submit 3 files: **the zip file, the doc file, and the pdf file** for Turnitin plagiarism check.
- Failure to submit the required pdf/doc file separately from the zip file will result in a 10% penalty (2.5 marks).
- Submissions that do not include the required pdf file will incur a 10% penalty (2.5 marks).
- Submissions that do not include the required work log doc file will incur a 20% penalty (5 marks).
- Submissions where the pdf file **fails the Turnitin plagiarism check**, **will not be graded**.
- Please verify your submission carefully before the deadline. If you submit the **wrong file**, a **corrupted zip file**, or **a submission that does not comply with the required file structure**, any resubmission made after the deadline will be subject to the standard late submission penalty.
- Late submission penalties are applied automatically by the system based on the assessment due date and the time of submission. **The penalty is determined by when you submit your work, not when you complete it.** Therefore, even if you completed the assessment before the due date, a late submission would still incur the applicable late penalty.
- Work must run on the **RMIT-provided EMR** cluster.
- **Please note that markers will not modify or adjust submitted code during marking.**

Functional Requirements

Failure to follow the requirements incurs up to 5 marks penalty.

- The code must be well written using a good coding style.
- The codes and scripts must come with concise and clear comments to explain the logical flow of the program.

Academic Integrity and Plagiarism (standard warning)

RMIT University treats plagiarism as a very serious offense constituting misconduct. Plagiarism covers a variety of inappropriate behaviours, including:

- Failure to properly document a source.
- Copyright material from the internet or databases
- Collusion between students

For further information on our policies and procedures, please refer to
https://www.rmit.edu.au/students/student-essentials/rights-and-responsibilities/academic-integrity

Assessment Criteria

Assessment for this MapReduce assignment will be based on two primary criteria:

1. **The correctness of the program's output**: the program must produce accurate and complete results as specified in the task requirements.
2. **The correctness of the task design and implementation**: your code should demonstrate a sound understanding of MapReduce principles, including
   - Efficient handling of large-scale BIG data,
   - Your implementation should leverage MapReduce's parallel processing model by correctly utilising multiple mappers and reducers to maximise scalability and performance.
   - Correct and efficient use of MapReduce components: mappers, reducers, combiners, partitioners, and data processing pipeline.

Marks will be awarded based on the quality of your design and implementation, rather than solely on producing the correct output. Solutions should demonstrate a sound understanding of MapReduce principles and make effective use of the framework's parallel processing capabilities.

Marking Guide

- Assignments received late and without prior extension approval or special consideration will be penalised by a deduction of 10% of the total score possible per calendar day late for that assessment.
- If unexpected circumstances affect your ability to complete the assignment, you can apply for special consideration.
  - Requests for special consideration within 7*24 hours, please email the course coordinator directly with supporting evidence.
  - Request for special consideration of more than 7*24 hours must be via the University Special consideration: https://www.rmit.edu.au/students/student-essentials/assessment-and-exams/assessment/special-consideration.

Task 1	0 marks - cannot run on AWS EMR or - no/unreasonable output or - >1 major logic error in the code	1 mark output incorrect due to 1 major logic error in the code or shell script	2-3 marks output incorrect due to >1 minor logic error in the code or shell script	4 marks output incorrect due to 1 minor logic error in the code or shell script	5 marks output correct and no code/script error, concise and clear comments
Task 2	0 marks - cannot run on AWS EMR or - no/unreasonable output or - >1 major logic error in the code	1-3 marks output incorrect due to 1 major logic error in the code or shell script	4-6 marks output incorrect due to >1 minor logic error in the code or shell script	7-8 marks output incorrect due to 1 minor logic error in the code or shell script	9-10 marks output correct and no code/script error, concise and clear comments
Task 3/4	0 marks - cannot run on AWS EMR or - no/unreasonable output or - >1 major logic error in the code	1-3 marks output incorrect due to 1 major logic error in the code or shell script	4-6 marks output incorrect due to >1 minor logic error in the code or shell script	7-8 marks output incorrect due to 1 minor logic error in the code or shell script	9-10 marks output correct and no code/script error, concise and clear comments
Functional requirement	Failure to follow the specified functional requirements will incur a penalty, as outlined in the assessment rubric.				
Format requirement	Failure to follow the specified formatting requirements will incur a penalty, as outlined in the assessment rubric.				

Marking in Practice

- Please thoroughly test your program prior to submission, as markers are not expected to modify any code (.py or .sh) you submit. For example, if your Bash script contains improper comments that prevent it from running, you will lose marks.
- The data files provided in this assessment do not represent big data; they are intended solely to verify the accuracy of your program's output. Nonetheless, always design your program with the potential to handle real BIG data scenarios in mind. The marker may test your code with a larger data set.